

TRABAJO DE UML

12 DIAGRAMAS UML Y CASOS DE USO

Presentado por:

Sebastian Antonio López Mojica

Docente:

Neider Hernandez

Institución:

Institucion Universitaria Del Caribe

Programa:

1. Introducción

El Lenguaje Unificado de Modelado, conocido como UML (Unified Modeling Language), es una herramienta utilizada en el desarrollo de software para representar, analizar, diseñar y documentar sistemas. UML permite visualizar la estructura de un sistema y la manera en que sus diferentes componentes interactúan entre sí, facilitando la comunicación entre desarrolladores, analistas, diseñadores y usuarios.

Los diagramas UML son representaciones gráficas que permiten estudiar diferentes aspectos de un sistema. Algunos permiten representar la estructura estática del software, mientras que otros muestran su comportamiento y la interacción entre los diferentes elementos.

Los casos de uso son igualmente importantes porque permiten describir las funciones que ofrece un sistema desde el punto de vista de sus usuarios. A través de ellos se pueden identificar los actores que interactúan con el sistema y las acciones que estos realizan.

2. ¿Qué es UML?

UML significa Unified Modeling Language (Lenguaje Unificado de Modelado). Es un lenguaje estándar utilizado para visualizar, especificar, construir y documentar sistemas de software.

UML no es un lenguaje de programación. Su función principal es servir como medio de comunicación y representación gráfica para comprender cómo funciona un sistema antes y durante su desarrollo.

Mediante UML es posible representar:

- La estructura de un sistema.
- Las relaciones entre sus componentes.
- El comportamiento del sistema.
- La interacción entre usuarios y aplicaciones.
- Los procesos y actividades.
- El flujo de información.
- La arquitectura y distribución del software.

3. Clasificación de los diagramas UML

Los diagramas UML se pueden dividir principalmente en dos grandes grupos: diagramas estructurales y diagramas de comportamiento. En este trabajo se presentan doce diagramas de uso frecuente en el ámbito académico.

Diagramas estructurales:

- Diagrama de clases.
- Diagrama de objetos.
- Diagrama de componentes.
- Diagrama de estructura compuesta.
- Diagrama de paquetes.
- Diagrama de despliegue.

Diagramas de comportamiento:

- Diagrama de casos de uso.
- Diagrama de actividades.
- Diagrama de estados.
- Diagrama de secuencia.
- Diagrama de comunicación.
- Diagrama de interacción general.

4. Diagrama de clases

El diagrama de clases representa las clases que forman parte de un sistema, sus atributos, métodos y las relaciones existentes entre ellas. Una clase generalmente se

representa mediante un rectángulo dividido en tres secciones: nombre de la clase, atributos y métodos u operaciones.

Ejemplo: clase Cliente

Atributos: id, nombre, correo, teléfono

Métodos: registrar(), actualizar(), eliminar()

Las clases pueden relacionarse mediante asociación, herencia, agregación, composición y dependencia. Este diagrama se utiliza principalmente durante el diseño de software orientado a objetos, ya que permite determinar cómo estará organizada la estructura interna del sistema.

5. Diagrama de objetos

El diagrama de objetos representa instancias concretas de las clases en un momento determinado. Mientras el diagrama de clases representa una estructura general, el diagrama de objetos muestra ejemplos reales de esas clases.

Ejemplo

cliente1: Cliente

nombre = Sebastian

correo = sebastian@gmail.com

Los diagramas de objetos sirven para comprender mejor cómo se comportan las clases cuando existen objetos específicos dentro del sistema.

6. Diagrama de componentes

El diagrama de componentes representa los diferentes componentes de software de un sistema y la forma en que se relacionan. Un componente puede ser un módulo, una biblioteca, una API, un servicio o un archivo ejecutable.

Ejemplo: sistema empresarial

Módulo de usuarios

Módulo de clientes
Módulo de inventario
Módulo de ventas
Base de datos

Permite comprender la arquitectura del software y facilita el mantenimiento y organización de sistemas grandes.

7. Diagrama de estructura compuesta

El diagrama de estructura compuesta muestra la estructura interna de una clase, componente o elemento complejo y las relaciones entre sus partes internas. Permite observar cómo un elemento está formado internamente.

Ejemplo: SistemaVentas

Módulo de productos
Módulo de clientes
Módulo de facturación

Es útil cuando se necesita estudiar la estructura interna de componentes complejos y sus conexiones.

8. Diagrama de paquetes

El diagrama de paquetes organiza los elementos del sistema en grupos llamados paquetes. Los paquetes permiten dividir un sistema en diferentes áreas para facilitar su organización.

Ejemplo: paquetes de un sistema

Usuarios
Clientes
Inventario
Ventas
Finanzas

Producción

Ayuda a organizar sistemas grandes y facilita la comprensión de la arquitectura general del software.

9. Diagrama de despliegue

El diagrama de despliegue representa cómo se distribuye físicamente el software en equipos y dispositivos. Muestra elementos como computadores, servidores, dispositivos móviles, bases de datos y redes.

Ejemplo

Computador del usuario → Servidor web → Servidor de aplicaciones → Base de datos

Se utiliza para representar la infraestructura física y la distribución de los componentes de un sistema.

10. Diagrama de casos de uso

El diagrama de casos de uso representa las funcionalidades que proporciona un sistema y la interacción entre esas funcionalidades y los usuarios o sistemas externos. Es uno de los diagramas más importantes durante el análisis de requisitos.

Sus principales elementos son:

- Actor: persona, organización, dispositivo o sistema externo que interactúa con el sistema.
- Caso de uso: función o servicio que el sistema proporciona.
- Sistema: límite dentro del cual se encuentran los casos de uso.
- Relaciones: asociación, <<include>>, <<extend>> y generalización.

Ejemplo: sistema de ventas

Actor: Vendedor

Casos de uso: Iniciar sesión, Registrar cliente, Registrar producto, Realizar venta, Generar factura

11. Diagrama de actividades

El diagrama de actividades representa el flujo de un proceso o procedimiento. Es parecido a un diagrama de flujo, pero utiliza la notación UML.

Puede representar inicio, actividades, decisiones, procesos paralelos y finalización.

Ejemplo: proceso de una venta

Inicio → Seleccionar cliente → Seleccionar productos → Calcular total → Confirmar venta → Generar factura → Fin

Si el cliente no está registrado, puede existir una decisión: ¿Cliente registrado? Sí: continuar con la venta. No: registrar cliente.

12. Diagrama de estados

El diagrama de estados, también conocido como diagrama de máquina de estados, representa los diferentes estados por los que puede pasar un objeto durante su ciclo de vida.

Ejemplo: orden de producción

Creada → En producción → Terminada → Entregada
También puede existir la transición: Creada → Cancelada

Es útil para representar sistemas donde los objetos cambian de estado dependiendo de diferentes eventos.

13. Diagrama de secuencia

El diagrama de secuencia representa la interacción entre diferentes objetos a través del tiempo. Se concentra en quién realiza una acción, qué mensaje se envía y en qué orden ocurre cada operación.

Ejemplo: inicio de sesión

1. Usuario introduce usuario y contraseña.
2. Sistema recibe los datos.
3. Sistema consulta la base de datos.
4. Base de datos devuelve la información.
5. Sistema valida las credenciales.
6. Sistema permite o rechaza el acceso.

Permite comprender detalladamente cómo se comunican los diferentes componentes de un sistema.

14. Diagrama de comunicación

El diagrama de comunicación muestra la interacción entre objetos, enfocándose en las relaciones entre ellos y en los mensajes que intercambian. Aunque guarda relación con el diagrama de secuencia, se concentra más en la estructura y conexiones entre los objetos que en la línea temporal.

En una venta pueden interactuar Vendedor, Sistema de ventas, Cliente, Producto, Factura y Base de datos.

Permite representar cómo colaboran diferentes objetos para ejecutar una determinada operación.

15. Diagrama de interacción general

El diagrama de interacción general combina elementos de los diagramas de actividades y de interacción. Se utiliza para representar de manera general diferentes interacciones dentro de un sistema.

Puede incluir secuencias, decisiones, condiciones, flujos e interacciones entre objetos. Es útil para representar procesos complejos en los que existen diferentes secuencias de interacción.

16. ¿Qué es un caso de uso?

Un caso de uso describe una función o comportamiento que un sistema debe proporcionar a un actor para alcanzar un objetivo. Los casos de uso se utilizan principalmente para capturar los requisitos funcionales de un sistema.

Ejemplo

Caso de uso: Registrar producto

Objetivo: permitir que un usuario autorizado registre un nuevo producto en el inventario.

17. Elementos de un caso de uso

Un caso de uso puede incluir los siguientes elementos:

- Nombre: identifica la funcionalidad. Ejemplo: Registrar producto.
- Actor: persona o sistema que inicia o participa en el caso de uso. Ejemplo: Administrador.
- Objetivo: indica lo que se desea conseguir.
- Precondiciones: condiciones que deben cumplirse antes de iniciar.
- Flujo principal: procedimiento normal paso a paso.
- Flujos alternativos: situaciones diferentes al flujo principal.
- Postcondiciones: resultados esperados al finalizar.

18. Relaciones en los casos de uso

Asociación: representa la comunicación entre un actor y un caso de uso. Ejemplo: Administrador — Registrar producto.

<<include>>: indica que un caso de uso siempre necesita ejecutar otro. Ejemplo: Realizar venta incluye Validar cliente.

<<extend>>: indica que un caso de uso puede agregar comportamiento adicional bajo determinadas condiciones. Ejemplo: Realizar venta puede ser extendido por Aplicar descuento.

Generalización: representa una relación de herencia entre actores o casos de uso.

Ejemplo: Empleado puede especializarse en Vendedor, Administrador o Supervisor.

19. Ejemplo completo de caso de uso

Caso de uso: Registrar cliente

Actor principal: Administrador.

Objetivo: Registrar un nuevo cliente en el sistema.

Precondición: El administrador debe haber iniciado sesión.

Flujo principal: 1) Ingresar al módulo de clientes. 2) Seleccionar “Nuevo cliente”. 3) Completar formulario. 4) Validar información. 5) Confirmar registro. 6) Almacenar información. 7) Mostrar mensaje de éxito.

Flujo alternativo: si faltan datos obligatorios, solicitar completar la información; si el cliente ya existe, informar que el registro está duplicado.

Postcondición: El cliente queda registrado en el sistema.

20. Importancia de UML en el desarrollo de software

UML es importante porque permite visualizar los sistemas antes de implementarlos.

Esto ayuda a reducir errores, mejorar la comunicación entre los integrantes del proyecto y facilitar el mantenimiento del software.

- Permite comprender mejor los requisitos.
- Facilita la comunicación entre desarrolladores y usuarios.
- Ayuda a detectar errores de diseño.
- Permite documentar el sistema.
- Facilita el mantenimiento.
- Ayuda a planificar la arquitectura.
- Permite representar procesos y relaciones complejas.

En proyectos grandes, utilizar UML puede ahorrar tiempo y recursos debido a que permite identificar problemas antes de comenzar la programación.

21. Ejemplo aplicado a un sistema empresarial

Para comprender mejor los diagramas UML, se puede considerar un sistema de gestión para una empresa de publicidad. El sistema puede tener módulos de usuarios, clientes, productos, cotizaciones, pedidos, producción, entregas, inventario y finanzas.

Los actores podrían ser Administrador, Gerente, Vendedor, Personal de producción y Personal de entregas.

Algunos casos de uso serían: Iniciar sesión, Registrar cliente, Registrar producto, Crear cotización, Aprobar pedido, Gestionar producción, Actualizar inventario, Registrar entrega y Consultar finanzas.

Ejemplos de aplicación de los diagramas

Casos de uso: muestran qué funciones puede realizar cada usuario.

Clases: muestran entidades como Cliente, Producto, Pedido y Usuario.

Secuencia: muestra cómo interactúan los elementos al realizar una venta.

Actividades: muestra el flujo completo de un pedido desde su creación hasta la entrega.

Estados: representa estados del pedido: Pendiente → Aprobado → En producción → Listo → Entregado.

22. Conclusión

Los diagramas UML constituyen una herramienta fundamental para el análisis y diseño de sistemas de software. Cada diagrama permite estudiar un aspecto diferente, ya sea la estructura del sistema, el comportamiento, la interacción entre objetos o la distribución de los componentes.

Entre los diagramas más importantes se encuentran el diagrama de clases, objetos, componentes, estructura compuesta, paquetes, despliegue, casos de uso, actividades, estados, secuencia, comunicación e interacción general.

Los casos de uso tienen una función especialmente importante porque permiten identificar las necesidades de los usuarios y establecer claramente las funciones que debe proporcionar un sistema.

En conclusión, conocer UML permite desarrollar sistemas más organizados, comprensibles y fáciles de mantener. Además, proporciona una forma estándar de representar los requerimientos y diseños de software, facilitando el trabajo conjunto entre analistas, programadores, diseñadores y usuarios.

23. Bibliografía

- Object Management Group (OMG). Unified Modeling Language (UML) Specification.
- Booch, G., Rumbaugh, J. y Jacobson, I. The Unified Modeling Language User Guide.
- Pressman, R. S. Ingeniería del Software: Un enfoque práctico.
- Sommerville, I. Ingeniería de Software.
- Fowler, M. UML Distilled: A Brief Guide to the Standard Object Modeling Language.